



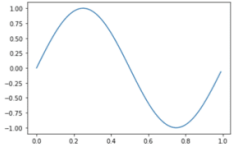
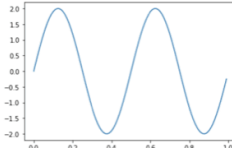
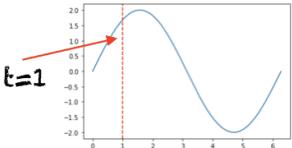
University
of Exeter

COMM117

Large Language Models and Application

Positional Encoding- How

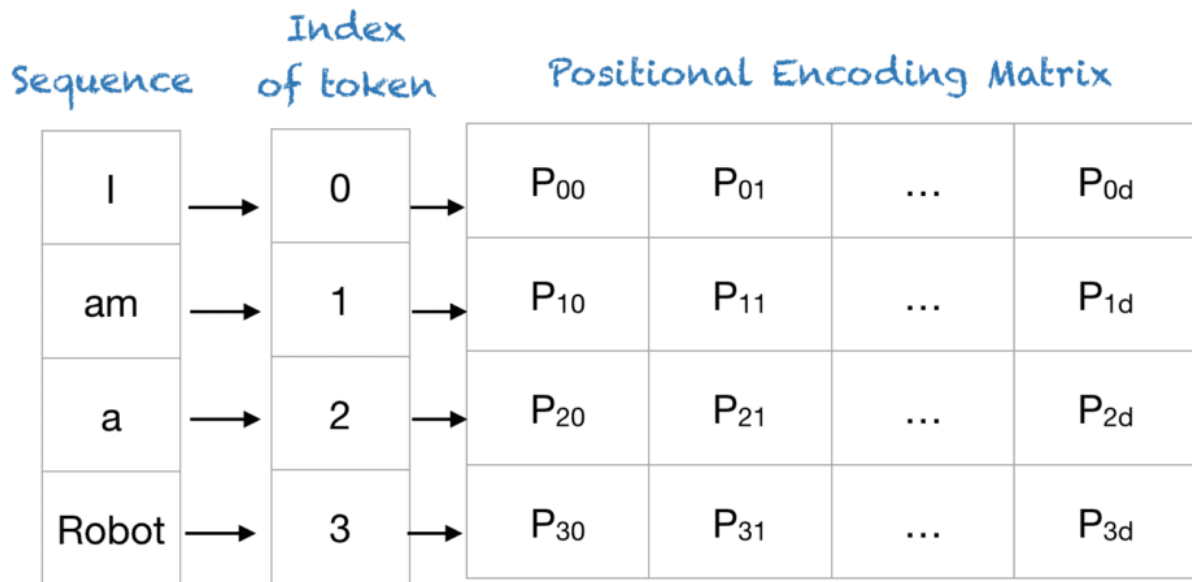
how quickly the curve goes up and down as you move along token positions.

Equation	Graph	Frequency	Wavelength
$\sin(2\pi t)$		1	1
$\sin(2 * 2\pi t)$		2	1/2
$\sin(t)$		$1/2\pi$	2π
$\sin(ct)$	Depends on c	$c/2\pi$	$2\pi/c$

By choosing many different c 's, we get multiple "position scales."

Positional Encoding- How

A Transformer uses **many sinusoids at different frequencies**, one per dimension.



Positional Encoding Matrix for the sequence 'I am a robot'

Morden Positional Encoding- LLM

➡ $PE(p) = [\sin(.), \cos(.), \sin(.), \cos(.), \dots]$ **Absolute PE**

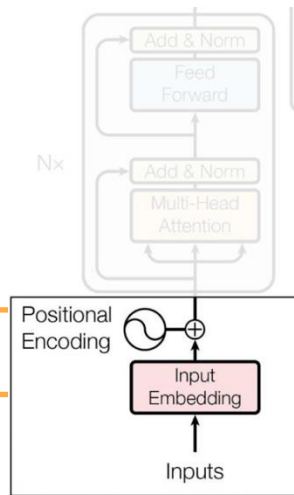
token embeddings $\rightarrow$ (+ **absolute PE**) $\rightarrow$ QKV



token embeddings $\rightarrow$ QKV $\rightarrow$ **Additive positional embedding K, V** $\rightarrow$ scores

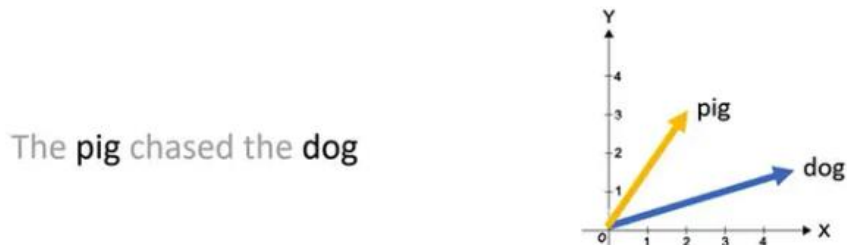


token embeddings $\rightarrow$ QKV $\rightarrow$ **apply RoPE to Q, K** $\rightarrow$ scores



Morden Positional Encoding- RoPE

- RoPE rotates Q and K by a position-dependent angle instead of adding positional vector.



Pros:

- **Stability of Vectors:** Adding tokens at the end of a sentence doesn't affect the vectors for words at the beginning, facilitating efficient caching.
- **Preservation of Relative Positions:** If two words, say “pig” and “dog,” maintain the same relative distance in different contexts, their vectors are rotated by the same amount.

Attention logits

$$\text{softmax} \left[S = \frac{QK^\top}{\sqrt{d_k}} + B_{\text{pos}} + M_{\text{mask}} \right]$$

Content-based similarity

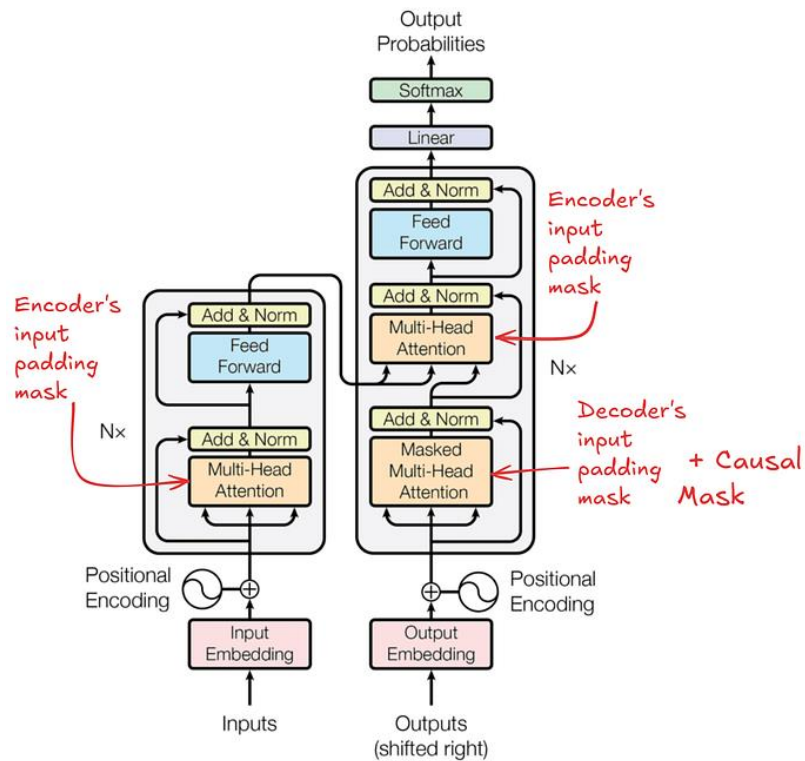
Positional effect

(relative bias /
RoPE effect
can be viewed
as modifying
geometry)

Hard constraint

Position is a preference/structure; masks are permissions

Masking

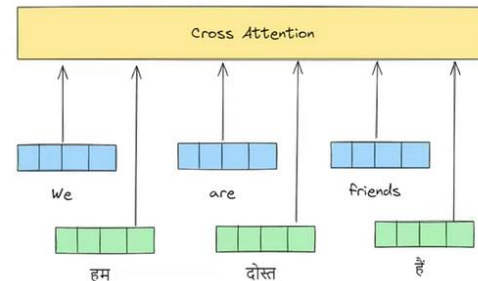


Encoder-Decoder

- **Encoder-only models (like BERT):** Use a bidirectional approach to understand context from both directions → **bidirectional self-attn**
- **Decoder-only models (like GPT, Llama):** These models process text from left to right and are particularly good at text generation tasks → **causal self-attn**
- **Encoder-decoder models (like T5, BART):** These models combine both approaches, using an encoder to understand the input and a decoder to generate output. → **encoder self-attn + decoder masked self-attn + cross-attn**

Encoder-Decoder- Cross attention

- **Machine Translation:** Translating text from one language to another involves comparing and aligning words between the source and target languages.
- **Question Answering:** Determining which parts of the context are relevant to answer a given question.
- **Image Captioning:** Generating descriptive text for images, where the image and text are treated as different modalities.
- **Text-to-Image Generation:** Creating images based on textual descriptions.
- **Text-to-Speech:** Generating speech from text, where the input is text and the output is speech.



Introduction Today

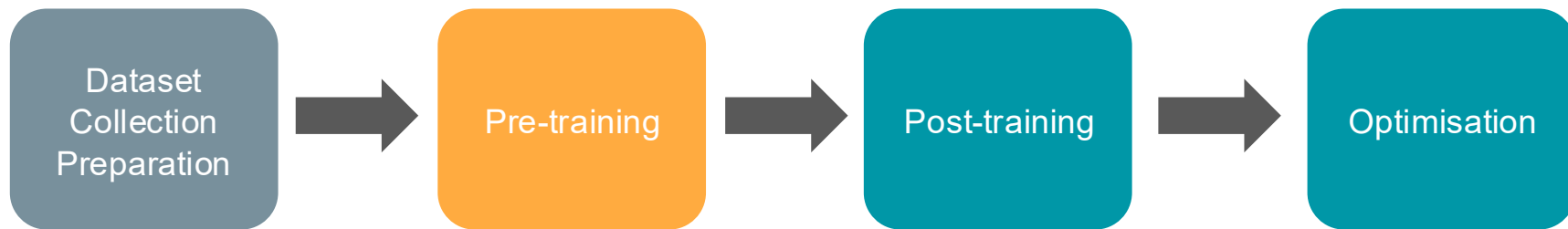
- Pretraining LLM
- Encoder, Decoder and Encoder-Decoder Objectives

Pre-training

- Data
- Transformer Architecture

How do we make it *learn*?

LLM training Pipeline



Pre-training

- Data
- Transformer Architecture

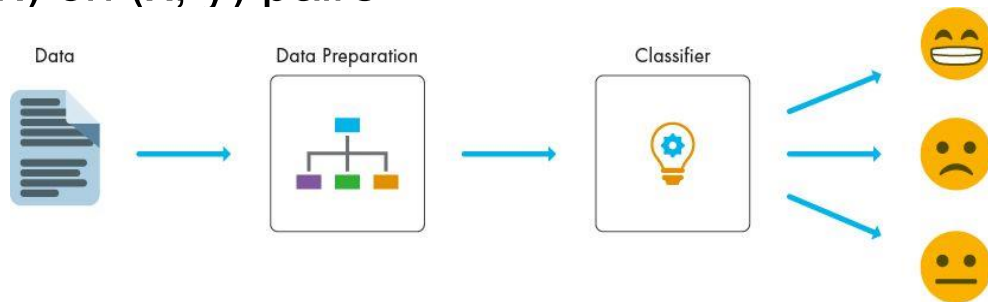
How do we make it *learn*?

Pretraining LLM: training a Transformer on a huge amount of **raw text** to learn **general language patterns by predicting tokens**, *before* any task-specific fine-tuning or alignment.

Traditional learning pipeline

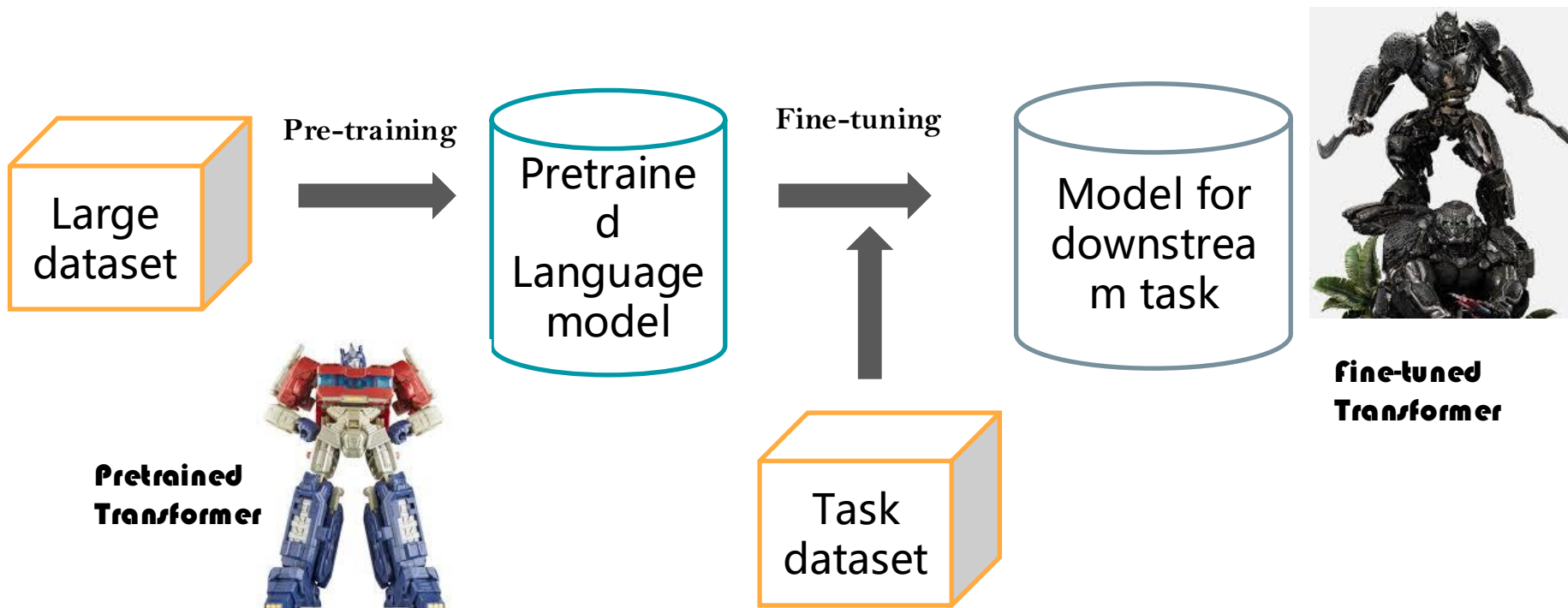
Supervised training/fine-tuning only, NO pre-training

- ✓ Collect (x, y) task training pairs
- ✓ Randomly initialize your models $f(x)$ (e.g., vanilla Transformers)
- ✓ Train $f(x)$ on (x, y) pairs

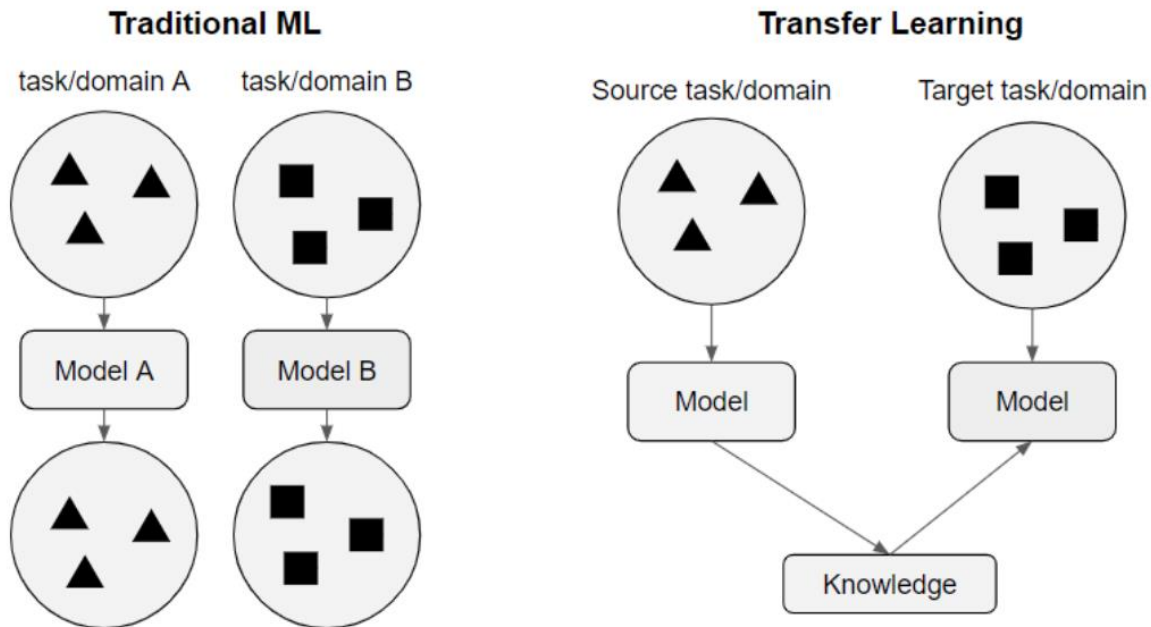


Then you get a trained Transformers ONLY for sentiment analysis
The model can be: RNNs, LSTM too

Classic NLP transfer learning (2018–2020)

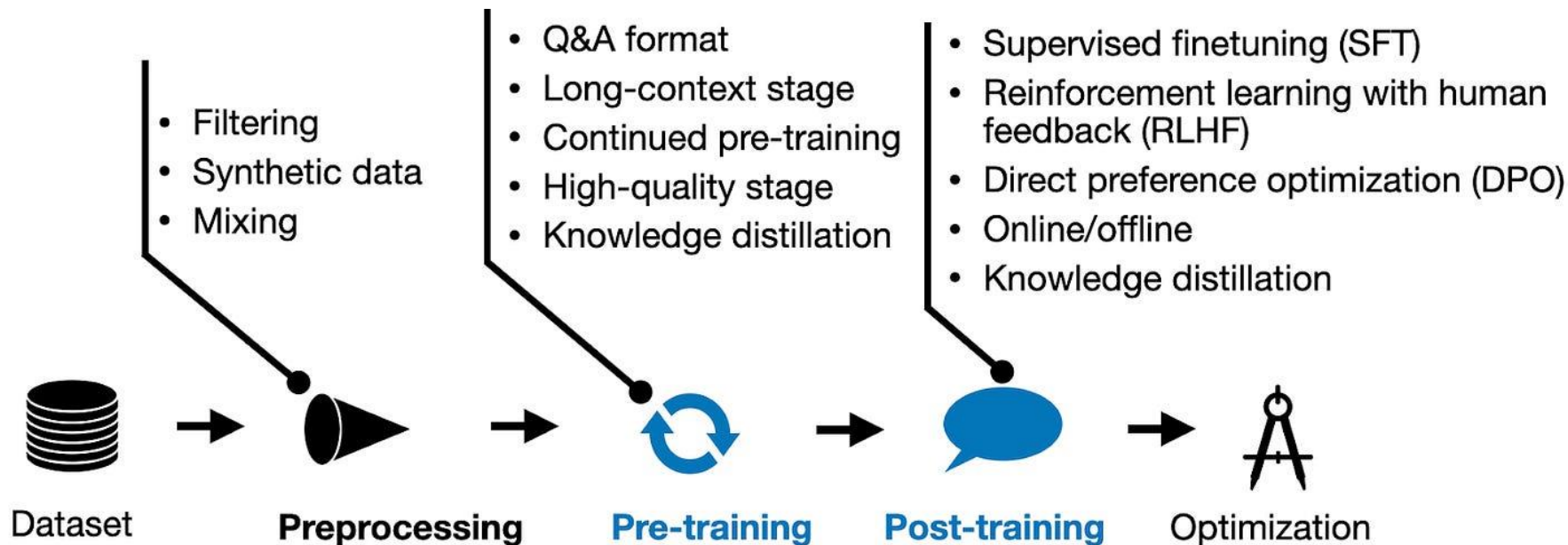


Classic NLP transfer learning (2018–2020)

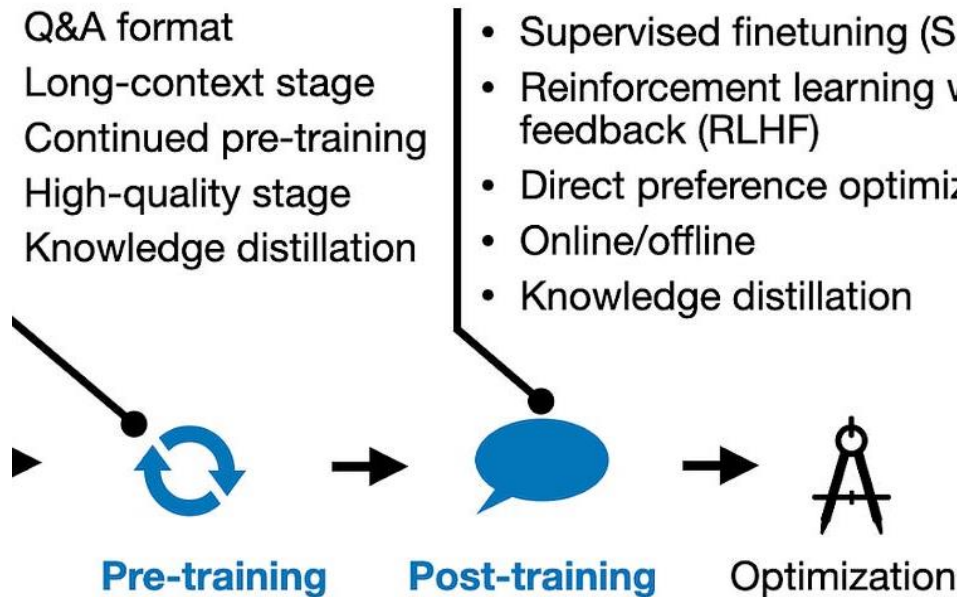


Knowledge gained in task A for source domain A is stored and applied to the problem of interest

Modern LLM post-training stack (2022–now)



Modern LLM post-training stack (2022–now)

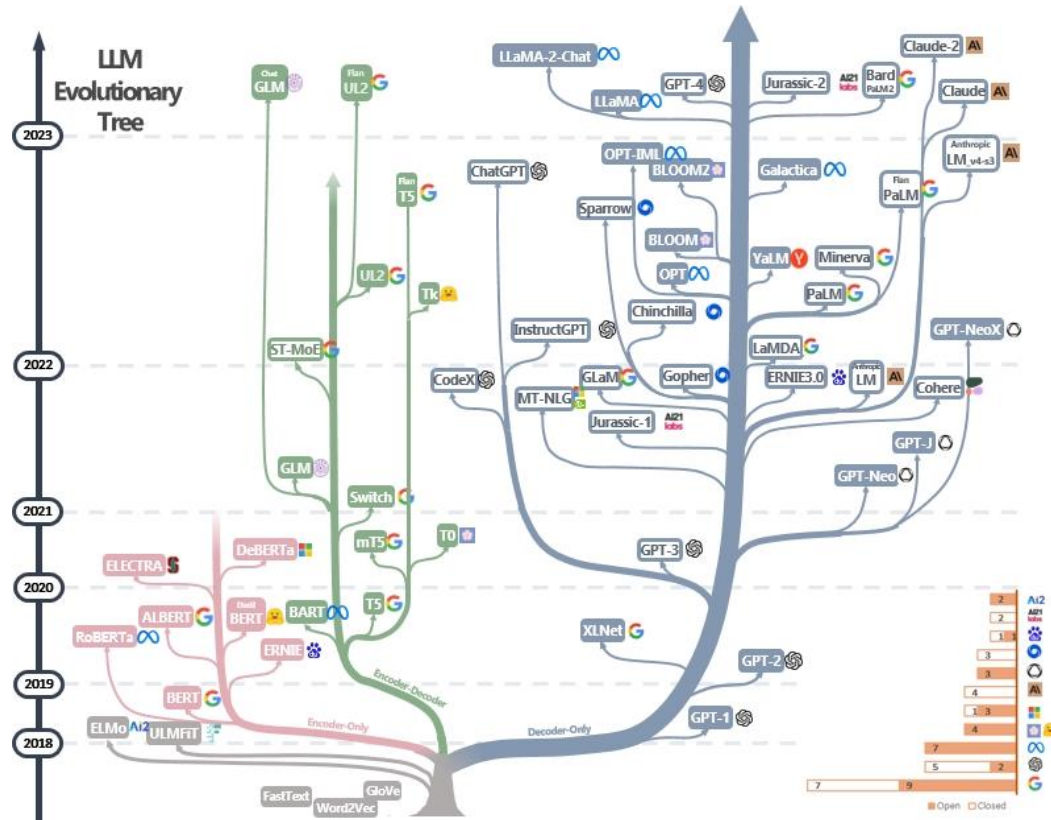


Output

general assistant / foundation model

- one model does many tasks via prompting
- post-training makes it helpful, safe, and instruction-following

Evolutionary tree of modern LLMs



<https://github.com/Mooler0410/LLMsPracticalGuide>

Pre-training LLMs

LLM pretraining and “transfer learning pretraining” are **the same core idea** (learn from large generic text first), but they differ in **goal, objective, and what comes next**.

- **LLM pretraining** trains a Transformer on massive text with a **self-supervised language modelling objective** (usually next-token prediction for decoder-only models)

Goal

- Learns a **general-purpose conditional distribution** over text

$$p_{\theta}(x_t | x_{<t})$$

Produces a **foundation model** that can:

Generate text; perform many tasks via **prompting** (in-context learning)

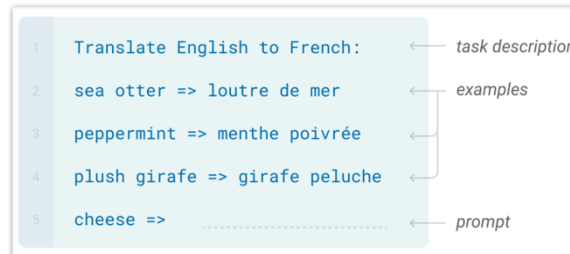
Prompting / In-context Learning

Pre-training → prompting/in-context learning (no training this step)

- Pretraining: Train a **large (>7~175B) Transformer** using **general text** using unsupervised learning.
- Prompting/in-context learning: Directly use the pretrained large Transformer (**no further finetuning/ training**) for any different task given only a **natural language description** of the task or a few task (x, y) examples → **Prompting**.



Zero-shot prompting



Few-shot prompting/in-context learning

Pretraining Objectives

Problem Definition

Input: raw text $\rightarrow$ tokens $x_{1:T}$

Output: model $p_{\theta}(\cdot)$

- Autoregressive factorization (decoder-only)

$$p_{\theta}(x_{1:T}) = \prod_{t=1}^T p_{\theta}(x_t | x_{<t})$$

Whole-sequence probability = multiply the conditional probabilities at each step.

Pretraining Objectives

Problem Definition

Input: raw text $\rightarrow$ tokens $x_{1:T}$

Output: model $p_{\theta}(\cdot)$

- Autoregressive factorization (decoder-only)

$$p_{\theta}(x_{1:T}) = \prod_{t=1}^T p_{\theta}(x_t | x_{<t})$$

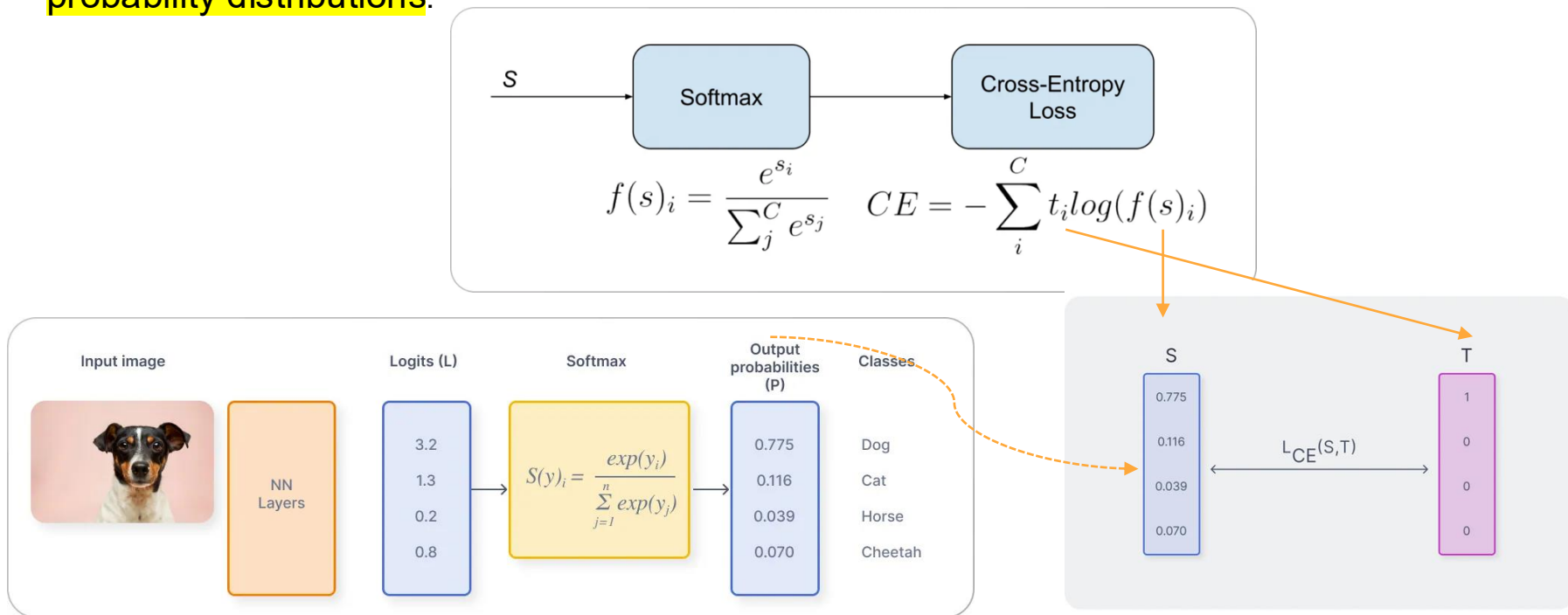
Maximum likelihood training

Choose θ to maximize
log-likelihood of dataset

$$\max_{\theta} \sum \log p_{\theta}(x_t | x_{<t})$$

Cross-entropy view

Cross entropy is a measure from information theory that quantifies the difference between two probability distributions.



Cross-entropy in LLM

Move from classes → next tokens

Next token prediction as classification task: Each potential word in the model's vocabulary acts as a separate class, and the model's task is to classify which word comes next given the context. → **more complex!**

cross entropy!

$$L_t = -\log p_{\theta}(y_t | x_{<t})$$

negative log-probability of the correct next token

Training minimizes average loss over all tokens in the batch:

$$L = \frac{1}{BN} \sum_{b,t} L_{b,t}$$

Teacher Forcing

- **Teacher forcing defines what inputs the model sees during training:** the ground-truth prefix tokens.
- **Cross-entropy defines what you optimize:** make the model assign high probability to the ground-truth next token.

Teacher forcing: feed the *true* previous tokens $x_{<t}$ to predict token x_t

In practice: **Shift-by-One Labels**

Inputs : the sequence **without the last token**

Targets : the same sequence **shifted left by 1 token**

- Input: "I am a"
- Target: "am a student"

Parallel training vs sequential generation

- Training: compute all token losses in parallel
- Inference: autoregressive sampling is sequential (dependency on sampled tokens)

Training (Teacher forcing)

- Ground-truth sequence is fully available
- Model predicts next-token distributions **for all positions in parallel**
- Compute loss at every position in one forward pass

Inference (Autoregressive decoding)

- Future tokens are **unknown**
- Each **generated token becomes part of the next input**
- Must decode one token at a time (sequential dependency)

Parallel training vs sequential generation

- Training: compute all token losses in parallel
- Inference: autoregressive sampling is sequential (dependency on sampled tokens)

Training (Teacher forcing)

Train contexts are from data distribution

- Ground-truth sequence is fully available
- Model predicts next-token distributions **for all positions in parallel**
- Compute loss at every position in one forward pass

Inference (Autoregressive decoding)

Infer contexts become model-generated

- Future tokens are unknown
- Each generated token becomes part of the next input
- Must decode one token at a time (sequential dependency)

However - Exposure bias

- Small error early → shifts context → compounding drift
- Longer generation amplifies

If model makes a rare-token mistake at step 5, what happens by step 200?

However - Exposure bias

If model makes a rare-token mistake at step 5, what happens by step 200?

Context drift (state moves off the training manifold)

Subsequent predictions are conditioned on a context with lower likelihood under the data distribution.

Error propagation / cascading

The mistaken token changes the semantics/syntax of the prefix → the model's next-token distribution shifts.

Mode collapse behaviours become more likely (common failure modes)

repetition loops, generic filler, topic derailment, or incoherent continuation
the model may “lock onto” high-probability **safe continuations** once uncertainty rises

safety

Why “rare token” matters

Rare tokens are **weakly estimated** and often **have noisier embeddings/gradients**.

However - Exposure bias

- We don't “solve” exposure bias in pretraining**
- **Pretraining prioritizes scalability and generality**
 - **Sequence-level methods come later (bridge to RLHF/DPO next lecture)**

Mode collapse behaviours become more likely (common failure modes)

repetition loops, generic filler, topic derailment, or incoherent continuation
the model may “lock onto” high-probability **safe continuations** once uncertainty rises

safety

Why “rare token” matters

Rare tokens are weakly estimated and often have noisier embeddings/gradients.

Pretraining Objectives



BERT (Encoder-only)

Devlin et al., 2018

The cabs ____ the same rates as those ____ by horse-drawn cabs and were ____ quite popular, ____ the Prince of Wales (the ____ King Edward VII) travelled in _____. The cabs quickly ____ known as "hummingbirds" for ____ noise made by their motors and their distinctive black and ____ livery. Passengers ____ the interior fittings were ____ when compared to ____ cabs but there ____ some complaints ____ the ____ lighting made them too ____ to those outside ____.

charged, used, initially, even, future, became, the, yellow, reported, that, luxurious, horse-drawn, were that, internal, conspicuous, cab



T5 (Encoder-decoder)

Raffel et al., 2019

Original text
Thank you for inviting me to your party last week.

Inputs
Thank you <X> me to your party <Y> week.

Targets
<X> for inviting <Y> last <Z>



Decoder-Only

Text: Second Law of Robotics: A robot must obey the orders given it by human beings

↓
Generated training examples

Example #	Input (features)	Correct output (labels)
1	Second law of robotics :	a
2	Second law of robotics : a	robot
3	Second law of robotics : a robot	must
...		

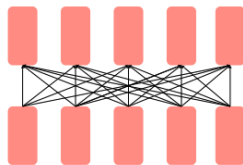
Masked token prediction Denoising span-mask prediction

Next Token Prediction

Pretraining Objectives- BERT



Encoder



- Bidirectional; can condition on the future context

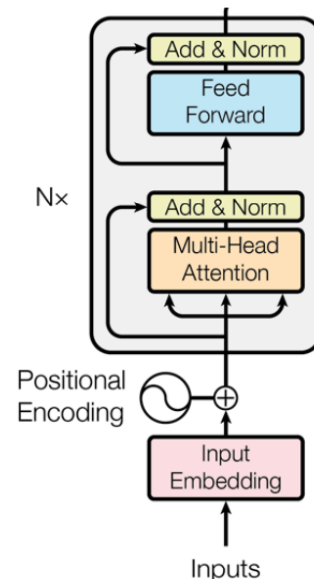
The cabs ____ the same rates as those ____ by horse-drawn cabs and were ____ quite popular, ____ the Prince of Wales (the ____ King Edward VII) travelled in _____. The cabs quickly ____ known as "hummingbirds" for ____ noise made by their motors and their distinctive black and ____ livery. Passengers ____ the interior fittings were ____ when compared to ____ cabs but there ____ some complaints ____ the ____ lighting made them too ____ to those outside ____.

charged, used, initially, even, future, became, the, yellow, reported, that, luxurious, horse-drawn, were that, internal, conspicuous, cab

Masked token prediction

New pre-training objectives:

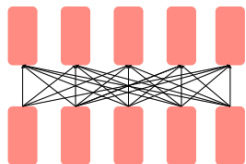
- **Masked language modeling (MLM)**
- **Next sentence prediction (NSP)** - Later work shows that NSP hurts performance though



Pretraining Objectives- Masked Language Modelling (MLM)

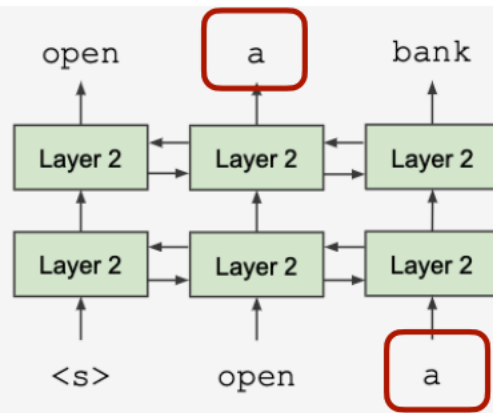
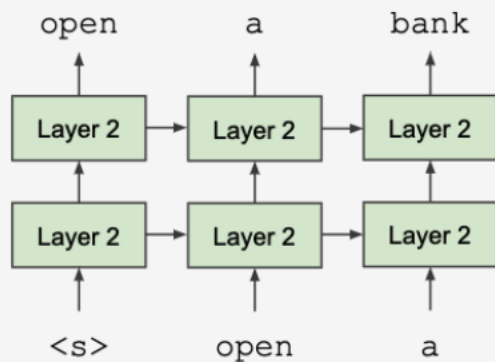


Encoder



- Bidirectional; can condition on the future context

we can't do language modelling with bidirectional models



Pretraining Objectives- Masked Language Modelling (MLM)



Solution:

Mask subset M , predict masked tokens using bidirectional context

$$\mathcal{L}_{MLM} = - \sum_{t \in M} \log p_{\theta}(x_t | x_{\setminus M})$$

store



gallon



$M = 2$

the man went to [MASK] to buy a [MASK] of milk

Pretraining Objectives- Masked Language Modelling (MLM)

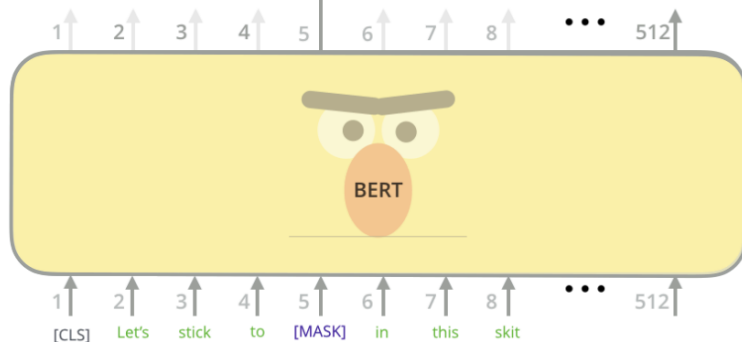


Use the output of the masked word's position to predict the masked word

Possible classes:
All English words

0.1%	Aardvark
...	...
10%	Improvisation
...	...
0%	Zyzzzyva

FFNN + Softmax



Randomly mask
15% of tokens

Input

[CLS] Let's stick to improvisation in this skit

<https://jalammar.github.io/illustrated-bert/>

Pretraining Objectives- Masked Language Modelling (MLM)



For the 15% predicted words,

- 80% of the time, they replace it with [MASK] token
went to the store → went to the [MASK]
- 10% of the time, they replace it with a random word in the vocabulary
went to the store → went to the running
- 10% of the time, they keep it unchanged
went to the store → went to the store

--Because [MASK] tokens are never seen during fine-tuning

Pretraining Objectives- Masked Language Modelling (MLM)

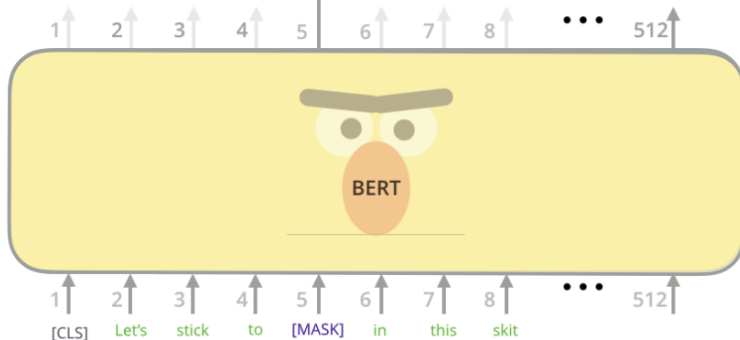


Use the output of the masked word's position to predict the masked word

Possible classes:
All English words

0.1%	Aardvark
...	...
10%	Improvisation
...	...
0%	Zyzzzyva

FFNN + Softmax



Randomly mask 15% of tokens

Input

[CLS] Let's stick to improvisation in this skit

Trade offs!

- Great representations;
weaker direct generation interface
- Mask token **distribution mismatch** -- normal text inputs **do not contain [MASK]!**

<https://jalammar.github.io/illustrated-bert/>

Pretraining Objectives- Next Sentence Prediction

- Motivation: Some tasks require understanding the relationship between two sentences (natural language inference, paraphrase detection, QA)
- NSP is designed to reduce the gap between pre-training and fine-tuning



Pretraining Objectives- Next Sentence Prediction



[CLS]: a special token
always at the beginning

[SEP]: a special token used
to separate two segments

Input = [CLS] the man went to [MASK] store [SEP]

he bought a gallon [MASK] milk [SEP]

Label = IsNext

Input = [CLS] the man [MASK] to the store [SEP]

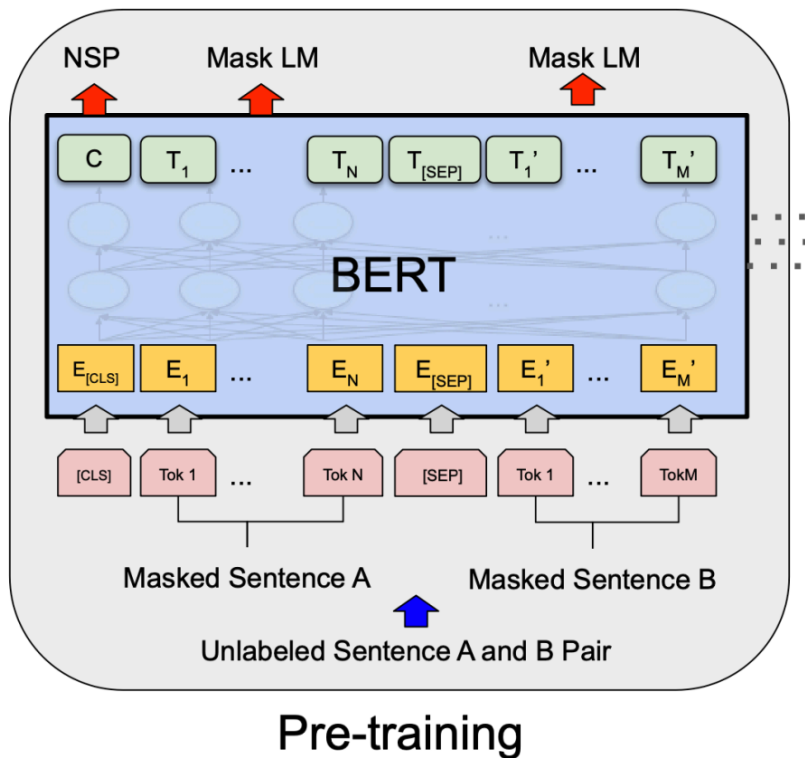
penguin [MASK] are flight ##less birds [SEP]

Label = NotNext

They sample two contiguous segments for 50% of the time and another random segment from the corpus for 50% of the time

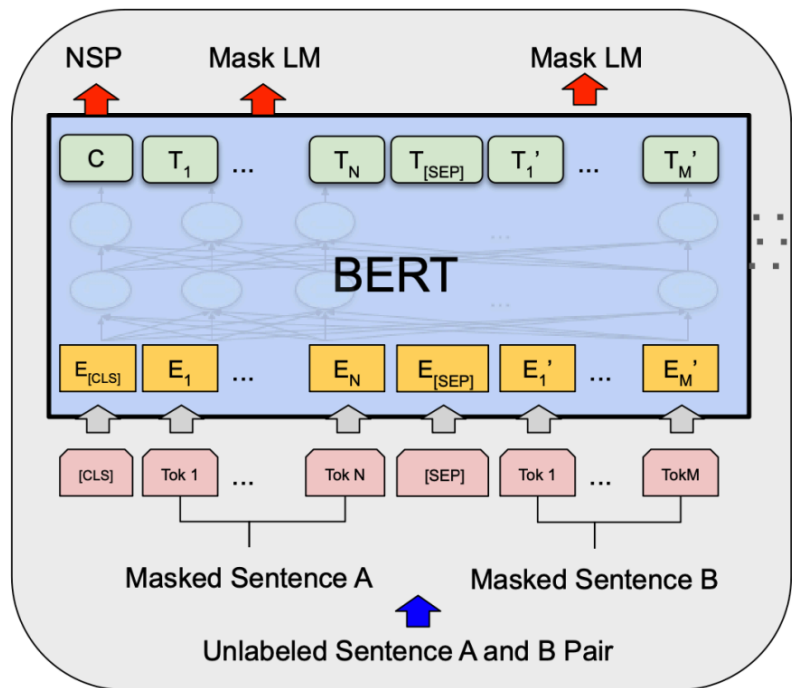
This actually hurts model learning based on later work!

Pretraining BERT-



- MLM and NSP are trained together
- [CLS] is pre-trained for NSP
- Other token representations are trained for MLM

Pretraining BERT-



Pre-training

Pros

- Consider both left and right context
- Capture intricate contextual relationships

Cons

- Not good at generating open-text from left-to-right, one token at a time

BERT MLM vs Denoising (Span Corruption)

From BERT MLM to Denoising Objectives

Denoising / Span Corruption (Encoder–Decoder)

Replace **different-length spans** from the input with unique placeholders (e.g., <extra_id_0>); decode out the masked spans

Natural fit for **seq2seq generation**

BERT MLM vs Denoising (Span Corruption)

From BERT MLM to Denoising Objectives

The capital of France is Paris.

BERT MLM

The capital of France is [MASK].

Target:

“Paris” (*only that token*)

Span corruption (span-level, T5-style)

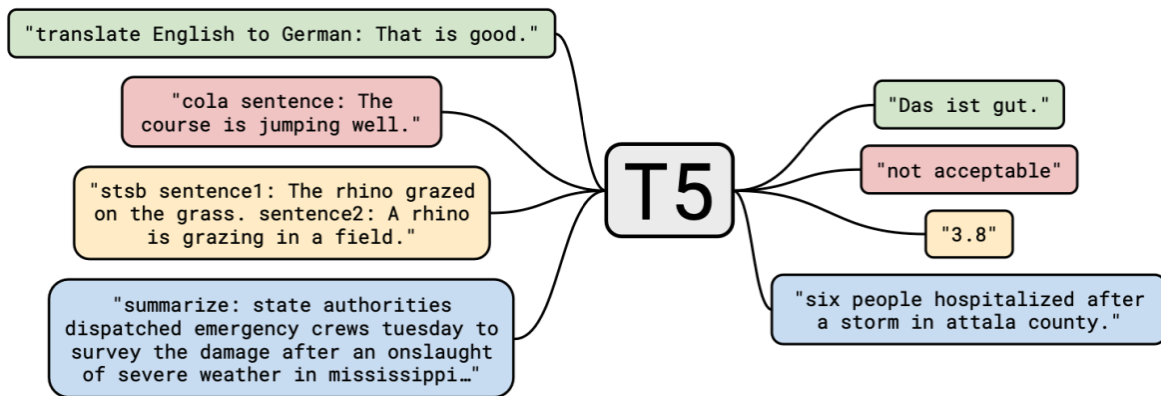
The capital of France is <extra_id_0>.

“<extra_id_0> Paris”

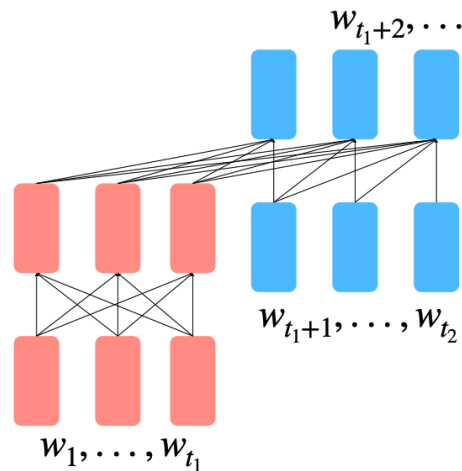
MLM predicts masked **tokens in place**;
denoising predicts missing content as a **generated sequence**.

Encoder-decoder: T5

T5 = Text-to-Text Transfer Transformer



- **Encoder** builds a representation of the source and gives it to the decoder
- **Decoder** uses the source representation to generate the target sentence



Encoder-decoder Training Objective

Original text

Thank you ~~for inviting~~ me to your party ~~last~~ week.

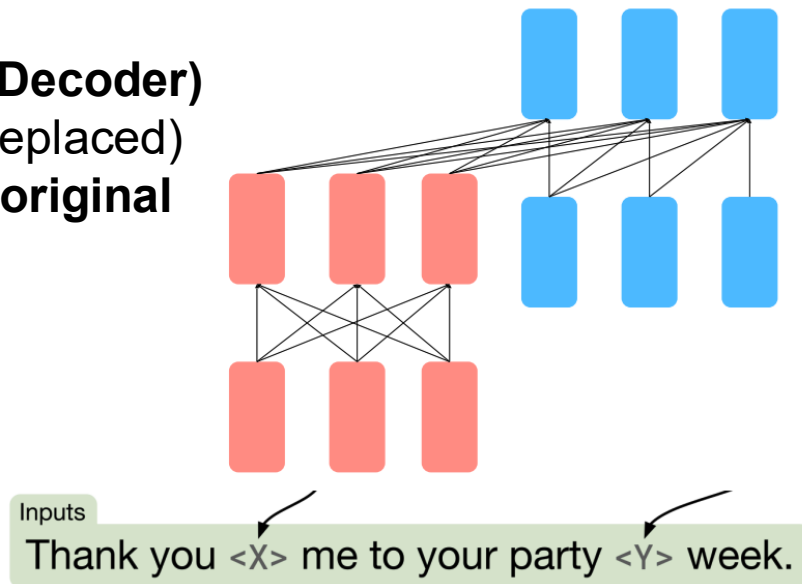
Targets

<X> for inviting <Y> last <Z>

Denoising / Span Corruption (Encoder-Decoder)

- Input: **corrupted text** (spans removed/replaced)
- Output: **generate the missing spans / original text**
- Objective:

$$\mathcal{L}_{denoise} = -\log p(x | \tilde{x})$$



Overview

Model family	Common pretraining objective	Self-attention mask	Cross-attention	Typical use
Encoder-only (BERT-style)	MLM (masked token prediction)	Padding mask only (bidirectional)	N/A	representation learning, classification, retrieval
Decoder-only (GPT/LLM-style)	AR (next-token prediction)	Causal mask + padding mask	N/A	text generation, chat, code
Encoder-decoder (T5/BART-style)	Denoising / span corruption (seq2seq)	Encoder: padding only ; Decoder: causal + padding	Yes: encoder outputs attended by decoder (padding mask on encoder)	translation, summarization, conditional generation

Advantages of Pre-training

- **Leveraging rich underlying information** from abundant raw texts.
- **Reducing the reliance of task-specific labeled data** that is difficult or costly to obtain.
- **Initializing model parameters** for more **generalizable** NLP applications.
- **Saving training cost** by providing a reusable model checkpoints.
- **Providing robust representation** of language contexts.

Next steps

- Training LLM II:

Training pipeline as an engineering system

Parallel training

Post training

Additional Material

1. Modern Approaches in Natural Language Processing (2020)
2. <https://jalammar.github.io/illustrated-bert/>
3. https://lenavoita.github.io/nlp_course/seq2seq_and_attention.html

Thank you!

If you want to ask us any questions about the module, the university, or anything else, drop us an email.

Pro tip: write **COMM117** on the email subject line, to make sure we give it priority.



r.mu2@exeter.ac.uk

Questions?